

Kryteria akceptacji i scenariusze testowe (GWT)

Projekt: KrysukEngine (Vector3D)
Lekki, przenośny silnik gier 3D z architekturą ECS

Karyna Rudenko (team leader)
Viktor Zasimovych

17 kwietnia 2026

Abstract

Niniejszy dokument zawiera komplet kryteriów akceptacji oraz scenariuszy testowych w formacie GWT dla historyjek użytkownika zdefiniowanych w dokumencie wizji projektu KrysukEngine. Dokument obejmuje cztery epiki: Podstawowy silnik ECS, Renderer 3D, Obsługę assetów oraz Przenośność między platformami.

1 Epik 1: Podstawowy silnik ECS

H1: Tworzenie encji i dodawanie komponentów

Jako deweloper chcę tworzyć encje i dodawać do nich komponenty, aby móc budować obiekty gry.

Kryteria akceptacji:

- System umożliwia utworzenie nowej, unikalnej encji (Entity) w ramach ECS Core.
- Do istniejącej encji można dodać jeden lub więcej komponentów (np. Transform, Mesh).
- Każda encja może posiadać maksymalnie jeden komponent danego typu (brak duplikatów tego samego typu komponentu).
- Próba dodania komponentu do nieistniejącej encji zwraca błąd / wyjątek.
- Dane komponentu są odczytywalne natychmiast po jego dodaniu do encji.

Scenariusz testowy 1.1 – Utworzenie encji i dodanie komponentu

Given system ECS Core jest zainicjalizowany i nie zawiera żadnych encji.

When deweloper wywołuje funkcję utworzenia nowej encji.

Then system zwraca unikalny identyfikator nowej encji.

And deweloper dodaje do tej encji komponent Transform.

Then komponent Transform jest powiązany z encją i dostępny do odczytu.

Scenariusz testowy 1.2 – Próba dodania duplikatu komponentu

Given encja posiada już przypisany komponent Transform.

When deweloper próbuje dodać do tej samej encji kolejny komponent typu Transform.

Then system odrzuca operację i zwraca informację o konflikcie typu komponentu.

Scenariusz testowy 1.3 – Dodanie komponentu do nieistniejącej encji

Given identyfikator encji nie istnieje w systemie (np. została usunięta).

When deweloper próbuje dodać komponent do tego identyfikatora.

Then system zgłasza błąd i nie modyfikuje stanu wewnętrznego ECS.

H2: Definiowanie własnych systemów

Jako deweloper chcę definiować własne systemy, które przetwarzają encje z określonymi komponentami.

Kryteria akceptacji:

- Deweloper może zarejestrować własny system, deklarując zestaw komponentów wymaganych do przetwarzania.
- System podczas każdej aktualizacji otrzymuje wyłącznie encje posiadające zadeklarowany zestaw komponentów.
- Możliwe jest zarejestrowanie wielu systemów działających niezależnie od siebie.
- Kolejność wykonywania systemów jest deterministyczna (zgodna z kolejnością rejestracji lub konfiguracją priorytetów).
- Encje nieposiadające wymaganych komponentów są automatycznie pomijane przez system.

Scenariusz testowy 2.1 – Filtrowanie encji przez system

Given zarejestrowano system wymagający komponentów Transform i Velocity; w świecie istnieją trzy encje: dwie z oboma komponentami, jedna tylko z Transform.

When wywoływana jest aktualizacja (update) systemu.

Then system przetwarza wyłącznie dwie encje posiadające oba wymagane komponenty.

Scenariusz testowy 2.2 – Rejestracja wielu niezależnych systemów

Given zarejestrowano dwa systemy: `MovementSystem` i `RenderSystem`, każdy z innym zestawem wymaganych komponentów.

When silnik wykonuje pojedynczą iterację pętli głównej.

Then oba systemy zostają wywołane niezależnie, każdy operując tylko na właściwych dla siebie encjach.

H3: Cykl życia systemów

Jako deweloper chcę, aby silnik automatycznie zarządzał cyklem życia systemów (init, update, destroy).

Kryteria akceptacji:

- Metoda `init()` systemu jest wywoływana automatycznie jednokrotnie, przed pierwszym wywołaniem `update()`.
- Metoda `update()` jest wywoływana w każdej iteracji pętli głównej aplikacji.
- Metoda `destroy()` jest wywoływana automatycznie przy zamknięciu aplikacji lub usunięciu systemu.
- Kolejność wywołań jest zgodna ze schematem: `init` → `(update)*` → `destroy`.
- Błąd zgłoszony w `init()` zatrzymuje rejestrację danego systemu i jest logowany, nie przerywając działania innych systemów.

Scenariusz testowy 3.1 – Pełny cykl życia systemu

Given zarejestrowano nowy system `PhysicsSystem` z implementacją `init/update/destroy`.

When aplikacja zostaje uruchomiona, wykonuje trzy iteracje pętli głównej, a następnie zostaje zamknięta.

Then metoda `init()` zostaje wywołana raz, `update()` trzy razy, a `destroy()` raz, w tej kolejności.

Scenariusz testowy 3.2 – Błąd inicjalizacji systemu

Given system `AudioSystem` zgłasza wyjątek w metodzie `init()`.

When silnik uruchamia aplikację.

Then błąd zostaje zalogowany, system `AudioSystem` nie jest dodawany do aktywnej listy systemów, a pozostałe systemy działają bez zakłóceń.

2 Epik 2: Renderer 3D

H4: Rysowanie modeli 3D z obsługą shaderów

Jako deweloper chcę rysować modele 3D na ekranie z obsługą podstawowych shaderów.

Kryteria akceptacji:

- Renderer jest w stanie wyświetlić wczytany model 3D w oknie aplikacji.
- Renderer wspiera co najmniej jeden podstawowy typ shadera (np. vertex + fragment shader).
- Model jest renderowany zgodnie z transformacją (pozycja, rotacja, skala) zapisaną w jego komponencie `Transform`.
- Błąd kompilacji shadera jest zgłaszany w logu i nie powoduje awarii całej aplikacji.
- Klatka renderowania odświeża się w sposób ciągły (w ramach pętli głównej) bez zauważalnego zacinania na referencyjnej scenie testowej.

Scenariusz testowy 4.1 – Wyświetlenie modelu z poprawnym shaderem

Given model 3D został wczytany i powiązany z encją posiadającą komponent **Transform**; przypisano poprawny shader.

When renderer wykonuje klatkę rysowania.

Then model zostaje wyświetlony w oknie aplikacji w pozycji zgodnej z komponentem **Transform**.

Scenariusz testowy 4.2 – Błąd kompilacji shadera

Given przypisany do modelu shader zawiera błąd składniowy.

When renderer próbuje skompilować shader przy starcie aplikacji.

Then błąd kompilacji zostaje zalogowany z opisem przyczyny, a aplikacja kontynuuje działanie (np. używając shadera domyślnego lub pomijając obiekt).

H5: Ładowanie i nakładanie tekstur

Jako deweloper chcę ładować tekstury i nakładać je na modele.

Kryteria akceptacji:

- Renderer wspiera wczytanie tekstur w formacie PNG i JPG.
- Wczytana tekstura może zostać przypisana do co najmniej jednego modelu 3D.
- Tekstura jest poprawnie mapowana na powierzchnię modelu zgodnie z jego współrzędnymi UV.
- Próba wczytania tekstury w nieobsługiwanym formacie zgłasza zrozumiały błąd, bez przerywania działania aplikacji.
- Jedna tekstura może być wielokrotnie wykorzystana (współdzielona) przez wiele modeli bez ponownego wczytywania z dysku.

Scenariusz testowy 5.1 – Nałożenie tekstury PNG na model

Given plik tekstury `model.png` istnieje na dysku, a model posiada zdefiniowane współrzędne UV.

When deweloper wywołuje załadowanie tekstury i przypisuje ją do modelu.

Then model jest renderowany z prawidłowo nałożoną teksturą zgodną z mapowaniem UV.

Scenariusz testowy 5.2 – Nieobsługiwany format tekstury

Given deweloper próbuje wczytać plik tekstury w formacie `.bmp`, który nie jest wspierany.

When wywoływana jest funkcja ładowania tekstury.

Then system zwraca komunikat błędu o nieobsługiwanym formacie, a aplikacja nie ulega awarii.

Scenariusz testowy 5.3 – Współdzielenie tekstury między modelami

Given tekstura `brick.png` została już raz wczytana do pamięci i przypisana do modelu A.

When deweloper przypisuje tę samą teksturę do modelu B.

Then tekstura nie jest ponownie wczytywana z dysku, a oba modele wykorzystują tę samą instancję w pamięci.

H6: Sterowanie kamerą

Jako deweloper chcę sterować kamerą (obracanie, przybliżanie), aby testować scenę z różnych perspektyw.

Kryteria akceptacji:

- Kamera obsługuje rotację wokół co najmniej dwóch osi (np. yaw, pitch).
- Kamera obsługuje przybliżanie/oddalanie (zoom) w określonym, skonfigurowanym zakresie.
- Zmiana parametrów kamery jest natychmiast odzwierciedlana w renderowanym obrazie (w kolejnej klatce).

- Parametry kamery (pozycja, kierunek, FOV) są dostępne do odczytu przez inne systemy.
- Próba ustawienia zoomu poza dopuszczalnym zakresem jest automatycznie ograniczana (clamp) do wartości granicznych.

Scenariusz testowy 6.1 – Obrót kamery

Given kamera znajduje się w domyślnej orientacji skierowanej na scenę.

When deweloper wywołuje obrót kamery o określony kąt w osi yaw.

Then w kolejnej renderowanej klatce widok sceny jest obrócony zgodnie z podanym kątem.

Scenariusz testowy 6.2 – Przybliżenie poza dopuszczalny zakres

Given maksymalny dopuszczalny poziom przybliżenia kamery wynosi określoną wartość graniczną.

When deweloper próbuje przybliżyć kamerę powyżej tej wartości.

Then poziom przybliżenia zostaje automatycznie ograniczony do wartości maksymalnej, bez błędu krytycznego.

3 Epik 3: Obsługa assetów

H7: Ładowanie modeli 3D z plików OBJ

Jako deweloper chcę ładować modele 3D z plików OBJ, aby używać własnych zasobów.

Kryteria akceptacji:

- Manager assetów wczytuje poprawny plik .obj i tworzy z niego model gotowy do renderowania.
- Wczytany model zachowuje geometrię (wierzchołki, normalne, współrzędne UV) zgodną z plikiem źródłowym.
- Próba wczytania uszkodzonego lub niepoprawnego pliku .obj zwraca zrozumiały błąd.
- Próba wczytania pliku o nieistniejącej ścieżce zgłasza błąd "plik nie znaleziony" bez awarii aplikacji.
- Wczytany model jest dostępny do natychmiastowego przypisania do encji w ECS.

Scenariusz testowy 7.1 – Wczytanie poprawnego modelu OBJ

Given plik `character.obj` jest poprawnym, kompletnym plikiem modelu 3D.

When deweloper wywołuje funkcję wczytania modelu z podaną ścieżką.

Then Manager assetów zwraca obiekt modelu z poprawnie wczytaną geometrią, gotowy do przypisania do encji.

Scenariusz testowy 7.2 – Wczytanie pliku spod nieistniejącej ścieżki

Given podana ścieżka `assets/missing.obj` nie wskazuje na istniejący plik.

When deweloper wywołuje funkcję wczytania modelu.

Then system zgłasza błąd "plik nie znaleziony", a aplikacja kontynuuje działanie.

Scenariusz testowy 7.3 – Wczytanie uszkodzonego pliku OBJ

Given plik `broken.obj` zawiera niepoprawną, uszkodzoną strukturę danych.

When deweloper wywołuje funkcję wczytania modelu.

Then system zgłasza błąd parsowania z opisem przyczyny i nie tworzy niekompletnego modelu.

H8: Ładowanie tekstur z plików PNG/JPG

Jako deweloper chcę ładować tekstury z plików PNG/JPG.

Kryteria akceptacji:

- Manager assetów wczytuje pliki tekstur w formatach PNG i JPG.
- Wczytana tekstura zachowuje oryginalną rozdzielczość i kanały kolorów (w tym kanał alfa dla PNG, jeśli obecny).
- Próba wczytania tekstury spod nieistniejącej ścieżki zgłasza zrozumiały błąd.
- Wczytane tekstury są przechowywane w pamięci podręcznej (cache) Managera assetów w celu ponownego wykorzystania.
- Wczytywanie dużej liczby tekstur (np. całego katalogu assetów) nie blokuje na trwałe głównego wątku aplikacji w sposób uniemożliwiający jej zamknięcie.

Scenariusz testowy 8.1 – Wczytanie tekstury PNG z kanałem alfa

Given plik `icon.png` zawiera kanał przezroczystości (alfa).

When Manager assetów wczytuje ten plik.

Then wczytana tekstura zachowuje informację o przezroczystości, widoczną podczas renderowania.

Scenariusz testowy 8.2 – Ponowne użycie tekstury z cache

Given tekstura `ground.jpg` została wcześniej wczytana i znajduje się w cache Managera assetów.

When inny moduł żąda wczytania tej samej tekstury po tej samej ścieżce.

Then Manager assetów zwraca instancję z cache, bez ponownego odczytu z dysku.

4 Epik 4: Przenośność między platformami

H9: Ten samy kod na Windows i Linux

Jako deweloper chcę uruchomić tę samą grę na Windows i Linux bez zmian w kodzie.

Kryteria akceptacji:

- Kod źródłowy gry napisany przy użyciu KrysukEngine kompiluje się bez modyfikacji na Windows i na Linux.
- Funkcjonalność ECS, renderera oraz systemu wejścia działa identycznie (z dokładnością do różnic sprzętowych) na obu platformach.
- Proces budowania (build) na obu platformach jest udokumentowany i wymaga tych samych kroków konceptualnych (np. jedno polecenie budujące).
- Brak konieczności stosowania warunkowej kompilacji specyficznej dla platformy w kodzie gry napisanym przez dewelopera korzystającego z silnika.
- Wynik działania referencyjnej scenarii testowej (np. scena z modelem i kamerą) jest wizualnie zgodny na obu platformach.

Scenariusz testowy 9.1 – Kompilacja tego samego projektu na dwóch platformach

Given projekt gry korzystający z KrysukEngine posiada jeden, niezmienny zestaw plików źródłowych.

When projekt zostaje zbudowany najpierw na systemie Windows, a następnie na systemie Linux.

Then w obu przypadkach budowa zakończy się sukcesem bez konieczności modyfikacji kodu źródłowego gry.

Scenariusz testowy 9.2 – Zgodność zachowania sceny testowej

Given ta sama scena testowa (model, tekstura, kamera) jest uruchomiona na Windows i na Linux.

When aplikacja wykonuje identyczną sekwencję działań (np. obrót kamery o ten sam kąt).

Then wynik wizualny (pozycja kamery, renderowany obraz) jest zgodny na obu platformach, z dokładnością do różnic sprzętowych GPU.

H10: Granie na różnych platformach

Jako gracz chcę grać w gry stworzone w Vector3D na moim systemie operacyjnym (Windows/Linux).

Kryteria akceptacji:

- Gracz może zainstalować i uruchomić skompilowaną grę na systemie Windows bez dodatkowych zależności poza standardowymi bibliotekami systemowymi.
- Gracz może zainstalować i uruchomić skompilowaną grę na systemie Linux bez dodatkowych zależności poza standardowymi bibliotekami systemowymi.
- Gra uruchamia się i działa z zachowaniem wydajności wystarczającej do płynnej rozgrywki (bez zauważalnego spadku liczby klatek na sekundę względem wersji referencyjnej).
- Zapisy stanu gry / ustawienia (jeśli dotyczy) są kompatybilne pomiędzy uruchomieniami na obu platformach.
- Komunikaty o błędach krytycznych (np. brak wymaganej biblioteki graficznej) są zrozumiałe dla gracza i wskazują sposób rozwiązania problemu.

Scenariusz testowy 10.1 – Uruchomienie gry na Windows

Given gracz posiada skompilowaną wersję gry przeznaczoną dla Windows oraz system spełniający minimalne wymagania.

When gracz uruchamia plik wykonywalny gry.

Then gra startuje poprawnie i wyświetla scenę bez błędów krytycznych.

Scenariusz testowy 10.2 – Uruchomienie gry na Linux

Given gracz posiada skompilowaną wersję gry przeznaczoną dla Linux oraz system spełniający minimalne wymagania.

When gracz uruchamia plik wykonywalny gry.

Then gra startuje poprawnie i wyświetla scenę bez błędów krytycznych, analogicznie do wersji Windows.

Scenariusz testowy 10.3 – Brak wymaganej biblioteki graficznej

Given na systemie gracza brakuje wymaganej biblioteki graficznej (np. odpowiedniej wersji OpenGL).

When gracz próbuje uruchomić grę.

Then aplikacja wyświetla zrozumiały komunikat błędu wskazujący brakującą zależność, zamiast nieoczekiwanej awarii.

5 Podsumowanie

Dokument obejmuje **10 historyjek użytkownika** (H1–H10) pogrupowanych w **4 epiki**, dla których zdefiniowano łącznie kryteria akceptacji oraz po 2–3 scenariusze testowe w formacie Given–When–Then. Scenariusze obejmują zarówno przypadki pozytywne, jak i przypadki błędów / wartości granicznych, zgodnie z zakresem usług.